

Root Finding in One and Several Dimensions

Ehsan Al Twal

Student Number: 89614622

PHYS 410: Computational Physics

September 22, 2026

1 Introduction

Bisection is guaranteed to converge but only linearly; Newton's method converges quadratically but may diverge if started poorly. Problem 1 combines the two into a hybrid scheme and uses it to find all ten roots on $[0, 2]$ of

$$f(x) = 512x^{10} - 5120x^9 + 21760x^8 - 51200x^7 + 72800x^6 - 64064x^5 + 34320x^4 - 10560x^3 + 1650x^2 - 100x + 1. \quad (1)$$

Problem 2 generalizes Newton's method to d dimensions and applies it to

$$x^2 + y^4 + z^6 = 2, \quad \cos(xyz^2) = x + y + z, \quad y^2 + z^3 = (x + y - z)^2, \quad (2)$$

near $\mathbf{x}^{(0)} = (-1.00, 0.75, 1.50)$.

2 Theory and numerical approach

2.1 Bisection

If f is continuous on $[a_0, b_0]$ with $f(a_0)f(b_0) < 0$, the interval brackets a root. At each iteration the midpoint $x_n = \frac{1}{2}(a_n + b_n)$ is formed and whichever half preserves the sign change is retained. The bracket width obeys $b_n - a_n = 2^{-n}(b_0 - a_0)$, so

$$|x_n - x^*| \leq 2^{-(n+1)}(b_0 - a_0), \quad (3)$$

i.e. linear convergence at $\log_{10} 2 \approx 0.30$ digits per step. Note that (3) bounds the error by a contracting envelope but does not force it to decrease at every step, since the midpoint may land nearer the root at one iteration than at the next.

2.2 Newton's method

Linearizing f about x_n and requiring the linear part to vanish gives

$$\delta_n = -\frac{f(x_n)}{f'(x_n)}, \quad x_{n+1} = x_n + \delta_n, \quad (4)$$

and expanding about the root yields $e_{n+1} = [f''(x^*)/2f'(x^*)]e_n^2 + \mathcal{O}(e_n^3)$: convergence is quadratic provided $f'(x^*) \neq 0$ and x_0 is close enough. In d dimensions the same construction gives a linear system rather than an explicit formula,

$$J(\mathbf{x}_n) \boldsymbol{\delta}_n = -\mathbf{f}(\mathbf{x}_n), \quad \mathbf{x}_{n+1} = \mathbf{x}_n + \boldsymbol{\delta}_n, \quad J_{ij} = \frac{\partial f_i}{\partial x_j}, \quad (5)$$

which is solved at each step with MATLAB's backslash operator rather than by forming J^{-1} . Convergence remains quadratic near a root at which J is nonsingular. There is no d -dimensional analogue of a bracket, so nothing plays the role bisection plays in Problem 1.

2.3 Implementation

hybrid biseects until $b_n - a_n \leq \text{tol1} \cdot |x_n|$, then takes the final midpoint as the starting point for (4), iterating until $|\delta_n| \leq \text{tol2} \cdot |x_{n+1}|$. Both criteria are relative, as specified, with a floor on the denominator to cover a root at the origin. **newtond** applies (5) until $\|\boldsymbol{\delta}_n\| \leq \text{tol} \cdot \|\mathbf{x}_{n+1}\|$. Both routines carry iteration caps.

Initial brackets are found by brute force: $[0, 2]$ is sampled at $N = 10001$ points and each adjacent pair with $f(x_i)f(x_{i+1}) < 0$ is recorded. The spacing $\Delta x = 2 \times 10^{-4}$ is far below the smallest root separation ($\approx 9.7 \times 10^{-2}$), so no root is missed. For Problem 2 the system (2) is written in residual form and its Jacobian is hard-coded analytically.

2.4 Exact roots of the test polynomial

The leading coefficient $512 = 2^9$ identifies (1) as a Chebyshev polynomial, $f(x) = T_{10}(x - 1)$. Since T_{10} vanishes at $\cos[(2k - 1)\pi/20]$, the roots are known exactly,

$$x_k^* = 1 + \cos\left(\frac{(2k - 1)\pi}{20}\right), \quad k = 1, \dots, 10, \quad (6)$$

all real, simple and in $[0, 2]$, providing an independent check on the results. No such closed form is available for (2).

3 Results

3.1 Problem 1

With $\text{tol1} = 10^{-5}$ and $\text{tol2} = 10^{-10}$ the search found exactly ten brackets and the hybrid routine converged on each (Table 1). Every root agrees with (6) to a relative accuracy between 1.5×10^{-15} and 1.2×10^{-11} . Comparison against MATLAB's **roots**, an unrelated companion-matrix algorithm, gave a maximum discrepancy of 1.194×10^{-10} .

3.2 Problem 2

Starting from $\mathbf{x}^{(0)} = (-1.00, 0.75, 1.50)$ with $\text{tol} = 10^{-12}$, **newtond** converged in nine iterations to

$$(x, y, z) = (-0.577705133337, 0.447447204972, 1.084412371725), \quad (7)$$

with residual $\|\mathbf{f}\| = 2.5 \times 10^{-16}$, i.e. at machine precision. The relative update at each step was

k	Computed root	Exact root, Eq. (6)	$ f(x_k) $
1	0.012311659404862	0.012311659404862	1.110×10^{-16}
2	0.108993475811632	0.108993475811632	1.443×10^{-15}
3	0.292893218813453	0.292893218813453	1.421×10^{-14}
4	0.546009500260421	0.546009500260453	1.359×10^{-13}
5	0.843565534959918	0.843565534959769	1.117×10^{-12}
6	1.156434465039169	1.156434465040231	8.584×10^{-12}
7	1.453990499739761	1.453990499739547	6.252×10^{-12}
8	1.707106781194015	1.707106781186547	4.700×10^{-11}
9	1.891006524210485	1.891006524188368	2.481×10^{-10}
10	1.987688340592448	1.987688340595138	7.740×10^{-10}

Table 1: Roots of Eq. (1) on $[0, 2]$ from the hybrid algorithm, with $\text{tol1} = 10^{-5}$ and $\text{tol2} = 10^{-10}$, compared against the exact values (6) and shown with the residual at each computed root. The growth of $|f(x_k)|$ with k reflects cancellation in the evaluation of f , not a loss of accuracy in the roots; see Section 4.

n	$\ \delta_n\ /\ \mathbf{x}_n\ $	n	$\ \delta_n\ /\ \mathbf{x}_n\ $
1	2.042×10^{-1}	6	5.686×10^{-3}
2	1.345×10^{-1}	7	4.265×10^{-4}
3	4.913×10^{-2}	8	7.024×10^{-7}
4	1.935×10^{-1}	9	4.010×10^{-13}
5	9.521×10^{-2}		

The iteration is not monotone at first, growing by a factor of four between $n = 3$ and $n = 4$, but from $n = 6$ onward the exponent roughly doubles at each step, which is the signature of quadratic convergence.

4 Discussion and conclusions

Both routines performed as intended. The bisection phase reliably delivered an estimate inside Newton’s basin of convergence, after which Newton reached tol2 in two or three steps—against the seventeen further halvings bisection would have needed—and the d -dimensional solver converged to (7) with a machine-precision residual.

The one result that looks like a defect is the growth of $|f(x_k)|$ across Table 1, spanning seven orders of magnitude. Comparison with the exact roots shows this is not a loss of accuracy in the roots themselves. Two effects contribute. The residual and error are related by $|f(x)| \approx |f'(x^*)||x - x^*|$, so a steeper root gives a larger residual for the same positional accuracy; more importantly, evaluating (1) near $x = 2$ sums terms of magnitude $512 \times 2^{10} \approx 5 \times 10^5$ which must cancel to something of order unity. That cancellation costs ≈ 5.7 digits and imposes a floor of order $10^{-16} \times 5 \times 10^5 \approx 10^{-10}$ on the computed value of f , irrespective of how well x is known. The observed residuals sit at that floor, and the same conditioning explains the 10^{-10} disagreement with **roots**. It also caps any useful choice of tol2 at around 10^{-12} for the outermost roots; a Chebyshev-basis representation would avoid the problem entirely.

Problem 2 illustrates the fragility of Newton’s method away from a root: the relative update grows at $n = 4$ before recovering, so the supplied starting point is close enough for eventual

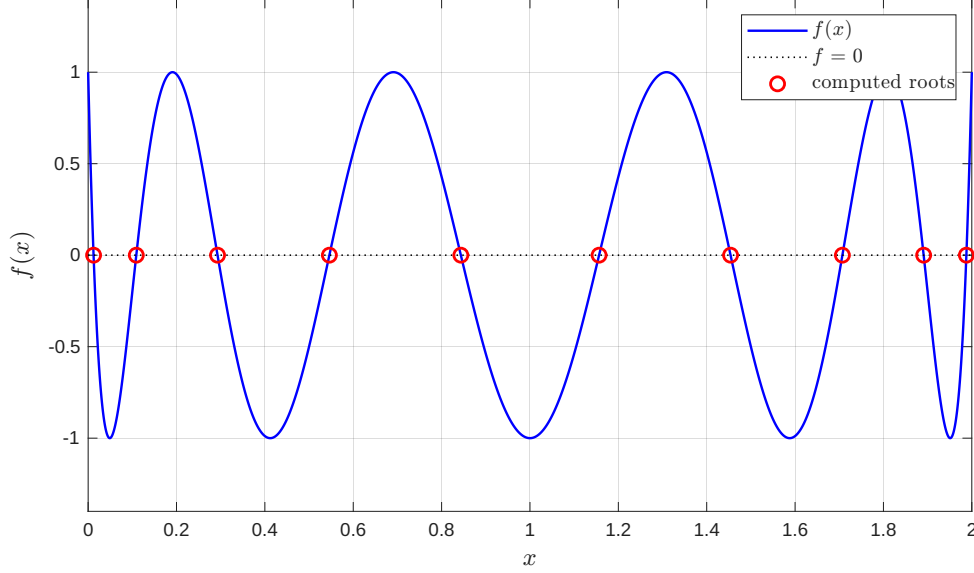


Figure 1: The test polynomial and its ten computed roots. Eq. (1) over the search interval $[0, 2]$, with the roots returned by the hybrid algorithm marked as circles. As $f(x) = T_{10}(x - 1)$ the function oscillates between ± 1 with ten simple zeros, clustered towards the ends of the interval in the characteristic Chebyshev distribution. Every sign change is marked and no marked point is off a sign change.

convergence but not for the quadratic law to hold from the outset. This is exactly what bisection guards against in Problem 1, and no analogous safeguard exists in several dimensions. Damping the step until the residual norm actually decreases would protect against the excursion seen here.

The main difficulty was interpreting the residuals of Table 1 before the identity $f(x) = T_{10}(x - 1)$ was noticed; without an independent reference it was unclear whether they indicated a genuine failure at the upper end of the interval or an artifact of the function evaluation. No comparable check exists for Problem 2, which is why the convergence history is reported: absent an exact answer, an observed quadratic rate together with a machine-precision residual is the strongest available evidence that the iteration has found a genuine root.

Statement on the use of generative AI. Generative AI (Anthropic’s Claude, via the Claude Code CLI) was used to draft the MATLAB implementations and the prose of this report, and analytically: the identification of (1) as $T_{10}(x - 1)$, and hence the closed form (6), came from that interaction, as did the cancellation argument above. All code was read, run and checked by me in MATLAB, and the numerical results reported here are the output of my own runs. The Problem 1 roots were verified independently against both (6) and MATLAB’s `roots`.

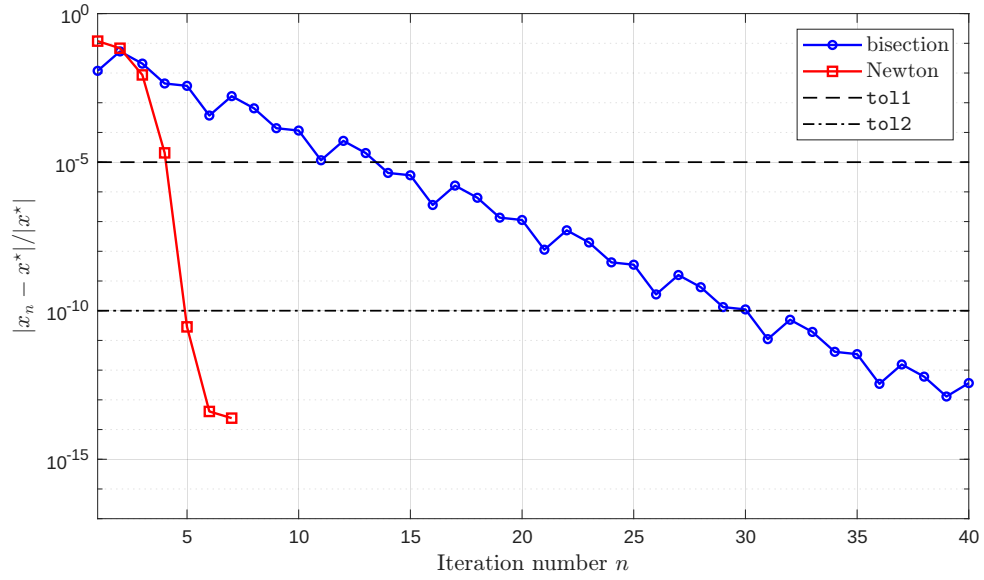


Figure 2: Linear versus quadratic convergence for the root near $x = 0.8436$. Relative error against iteration number for bisection and for Newton started from a comparable offset, with both tolerances marked. The bisection error descends raggedly, as expected from (3), which bounds the error by a 2^{-n} envelope without forcing a decrease at every step; averaged over the forty iterations shown it follows the predicted 0.30 digits per step. Newton instead squares the error at each step and reaches $\sim 10^{-14}$ by the fifth iteration. The flattening below $\sim 10^{-13}$ is the accuracy floor of the reference value used to form the error, not a property of either method.